

COMP2054-ADE:

ADE Lec04a: The Big-Oh family

Lecturer: Andrew Parkes

andrew.parkes 'at' Nottingham.ac.uk

<http://www.cs.nott.ac.uk/~pszajp/>

Relatives of Big-Oh

A close family:

- **big-Oh** $\mathcal{O}$
- **big-Omega** $\mathcal{\Omega}$
- **big-Theta** $\mathcal{\Theta}$

- **little-oh** $\mathcal{o}$
 - note this is not in the main text-book but is required for the module
- **little-omega** $\mathcal{\omega}$
 - Not required, as not used a lot, though mention it for completeness

**** Definition: “Big-Omega” ****

For two positive functions $f(n)$ and $g(n)$, of a single variable n , then

$f(n)$ is $\Omega(g(n))$ “ f is Big-Omega of g ”

if and only if there exist constants $c > 0$ and $n_0 \geq 1$ such that

for all $n \geq n_0$, $f(n) \geq c g(n)$

** Definition: “Big-Omega” **

For two positive functions $f(n)$ and $g(n)$, of a single variable n , then

$f(n)$ is $\Omega(g(n))$ “ f is Big-Omega of g ”

iff if there exist constants $c > 0$ and $n_0 \geq 1$ such that

for all $n \geq n_0$. $f(n) \geq c g(n)$

- Spot the big difference from big-Oh?
 - “greater than or equal” rather than “less than or equal”
- Note that need $c > 0$, and we not allowed $c=0$
 - If we were allowed $c=0$ then Omega would become useless
 - In Big-Oh, $c=0$ might be allowed or not but makes no significant difference either way (Exercise: think why)
 - As for Big-Oh, c must be **constant (cannot depend on n)**

Exercise (online):

- Show n is $\Omega(n)$

Need $c > 0$, n_0 such that

$$n \geq c n \text{ for all } n \geq n_0$$

Try $c = 1$

$$n \geq n \text{ for all } n \geq n_0$$

Pick $n_0=1$.

DONE.

Exercise (online):

- Show n^2 is $\Omega(n)$

Need $c > 0$, n_0 such that

$$n^2 \geq c n \text{ for all } n \geq n_0$$

Try $c = 1$

$$n \geq 1 \text{ for all } n \geq n_0$$

Pick $n_0 = 1$

Also works if try $c = 2$

$$n \geq 2 \text{ for all } n \geq n_0$$

$$n_0 = 2$$

DONE.

Exercise (online):

- Show $n^3 - n$ is $\Omega(n^3)$

Need $c > 0$, n_0 such that

$$n^3 - n \geq c n^3 \text{ for all } n \geq n_0$$

Try $c = 1$

$$n^3 - n \geq n^3 \text{ for all } n \geq n_0$$

$$-n \geq 0 \text{ for all } n \geq n_0$$

FAILS

Try $c = 1/2$

$$2n^3 - 2n \geq n^3 \text{ for all } n \geq n_0$$

$$n^2 \geq 2 \text{ for all } n \geq n_0$$

$n_0 = 2$ DONE.

Exercise (online):

- Is it true that: 1 is $\Omega(n)$?

Need $c > 0$, n_0 such that

$$1 \geq c n \text{ for all } n \geq n_0$$

Note: No need for c to be integer

Try $c = \dots$ FAILS

Instead use $c > 0$ to get

$$(1/c) \geq n \text{ for all } n \geq n_0$$

FAILS as eventually $n > (1/c)$

FAILS. That is

$$1 \text{ is NOT } \Omega(n)$$

Note: $c = 1/n$ is NOT allowed; need constant c

Big-Omega Examples

- We have
 - n is $\Omega(1)$
 - n is $\Omega(n)$
 - n is not $\Omega(n^2)$
- This is “the opposite of Big-Oh”
- **$f(n)$ is $\Omega(g(n))$ says that “ $f(n)$ grows at least as fast as $g(n)$ at large n ”**
- Compared to
 - $f(n)$ is $O(g(n))$ says that “ $f(n)$ grows no faster than $g(n)$ at large n ”

Big-Omega properties

- Similarly to big-Oh, Big-Omega is
 - Reflexive
 - NOT symmetric
 - Transitive
- Exercise (offline): Prove these.

Linking big-Oh and Big-Omega

- Suppose that we are give “f is $O(g)$ ”
- What can we say about big-Omega?

Linking big-Oh and Big-Omega

- Suppose that we are give “f is $O(g)$ ”
 - We know there exist c n_0 such that
$$f(n) \leq c g(n) \quad \forall n \geq n_0$$
 - We can assume $c > 0$ (Why?). Hence
 - $g(n) \geq \left(\frac{1}{c}\right) f(n) \quad \forall n \geq n_0$
 - Hence, g is $\Omega(f)$.
- That is: $f \in O(g) \rightarrow g \in \Omega(f)$
- Similarly: $f \in \Omega(g) \rightarrow g \in O(f)$.
- Note: similar to: $x \leq y \rightarrow y \geq x$

Usage of big-Omega

- Once familiar with big-Oh then big-Omega is very similar but “upside down”. But, same “quick rules” apply:
 - Multiplication rule still applies
 - Can still drop smaller terms
- E.g. $n^3 - n$ is $\Omega(n^3)$
- ‘ Ω ’ expresses “grows as least as fast as”
- ‘ O ’ expresses “grows at most as fast as”

** 'Paradox' of big-Omega **

- “Can still drop smaller terms”
- E.g. $n^3 + n$ is $\Omega(n^3)$
- Note that this might seem counter-intuitive
 - thinking “*big Omega does ‘at least’ and so need the smallest term*” is incorrect!
 - instead think of the large n behaviour being dominated by the n^3 , and that this grows at least as fast as n^3 .
 - This is a common error on assessments!

Omega Usage

- A standard usage might be to capture a limitation on the best one can hope for,
e.g. *"The best case for algorithm X is $\Omega(n^3)$ and so it will not scale well"*
- However, if the worst-case behaviour of an algorithm is a 'bizarre' or 'partially unknown' function of n , then one might say,
e.g. *"The worst case for algorithm X is not precisely known but we know it is $\Omega(n^3)$ and $O(n^4)$ "*
- Gives a lot more flexibility than doing ratios.

**** Definition: “Big-Theta” ****

For two positive functions $f(n)$ and $g(n)$, of a single variable n , then

$f(n)$ is $\Theta(g(n))$ “ f is Big-Theta of g ”

if and only if there exist constants $c_1, c_2 > 0$, and $n_0 \geq 1$ such that

forall $n \geq n_0$. $f(n) \leq c_1 g(n)$

and

forall $n \geq n_0$. $f(n) \geq c_2 g(n)$

**** Definition: “Big-Theta” ****

For two positive functions $f(n)$ and $g(n)$, of a single variable n , then

$f(n)$ is $\Theta(g(n))$ “ f is Big-Theta of g ”

if and only if there exist constants $c_1, c_2 > 0$, and $n_0 \geq 1$ such that

for all $n \geq n_0$. $f(n) \leq c_1 g(n)$ and

for all $n \geq n_0$. $f(n) \geq c_2 g(n)$

- *What does this say about the growth rate of $f(n)$?*
- *How does it connect to O and Ω ?*

**** Definition: “Big-Theta” ****

$f(n)$ is $\Theta(g(n))$ “f is Big-Theta of g”
iff there exist constants $c_1, c_2 > 0$, and $n_0 \geq 1$ s.t.
for all $n \geq n_0$. $f(n) \leq c_1 g(n)$ and
for all $n \geq n_0$. $f(n) \geq c_2 g(n)$

- $f(n)$ is $\Theta(g(n))$ if and only if
 $f(n)$ is $O(g(n))$ and also $f(n)$ is $\Omega(g(n))$
- Example: $2n+1$ is $\Theta(n)$
- Θ expresses “grows ‘exactly’ as fast as”
 - But with the caveat that “exactly” ignores constants – it does not imply $f(n) = g(n)$

EXERCISE

- Consider the function:

$$f(n) = \begin{cases} n & \text{if } n \text{ is even} \\ 2n & \text{if } n \text{ is odd} \end{cases}$$

What is its big-Omega behaviour?

Clearly is $\Omega(n)$.

Just take $c=1$ $n_0=1$ then

$$f(n) \geq 1 \cdot n \quad \text{for all } n \geq 1$$

EXERCISE

- Consider the function:

$$f(n) = \begin{cases} n & \text{if } n \text{ is even} \\ 2n & \text{if } n \text{ is odd} \end{cases}$$

What is its big-Theta behaviour?

Clearly is $\Theta(n)$. As is both $O(n)$ and $\Omega(n)$.

But note that there is not a single value for ratio $f(n)/n$.

$\Theta(n)$ expresses the growth rate is linear but does not tightly sandwich the specific values.

- This is very useful in CS !
- Quite often, people can really mean Big-Theta when they say Big-Oh

Big-Theta is reflexive and transitive

- Trivial as both O and Ω are.

Is Big-Theta symmetric?

- If $f \in \Theta(g)$ then it is (automatically and always) true that $g \in \Theta(f)$
- Exercise (offline): Prove this. (It follows quickly from previous results).

Θ is an equivalence relation

- Any relation that is
 - Reflexive & Symmetric & Transitive
- is an “equivalence relation”
 - Roughly speaking: it behaves like a “equality”:
 - $\Theta(g)$ is “the equivalence class of all functions whose large n behaviour is bounded above and below by constants times $g(n)$ ”
- It is “reasonable” to write $f = \Theta(g)$ “ $f = \Theta(g)$ ”
 - (Not needed: pedantically, arguably, could be better to write “ $\{f\} = \Theta(g)$ ” but dropping “ $\{\}$ ” on singleton sets is a common abuse of notation. Even then, it would still be an abuse of notation as it is not a set equality, rather it is an “equivalence relation” giving an “equivalence class”, denoted with “ $[\]$ ” i.e. closer to “ $[f] = \Theta(g)$ ”)
 - “Reasonable” is not same as “recommended” 😊
“ f is $\Theta(g)$ ” or “ $f \in \Theta(g)$ ” are clearer (IMHO)

Next Lecture

- “little-oh” $f(n)$ is $o(g(n))$